Code:

```
def calculate_heuristic(state, goal):
    misplaced = []
    for i in range(3):
        for j in range(3):
            if state[i][j] != 0 and state[i][j] != goal[i][j]:
                misplaced.append(state[i][j])
    return len(misplaced), misplaced

def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j
    return -1, -1

def get_neighbors(state):
    r, c = find_blank(state)
    moves = [
        ("Up", -1, 0),
        ("Down", 1, 0),
        ("Left", 0, -1),
        ("Right", 0, 1)
    ]
    neighbors = []
    for name, dr, dc in moves:
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            new_state = [row[:] for row in state]
            new_state[r][c], new_state[nr][nc] = new_state[nr][nc], new_state[r][c]
            neighbors.append((name, new_state))
    return neighbors

def print_state(state):
    for row in state:
        print(" ".join(str(x) if x != 0 else "_" for x in row))

def hill_climbing(initial_state, goal_state):
    current = initial_state
    step = 0
    current_h, misplaced = calculate_heuristic(current, goal_state)
    print("Initial State:")
    print_state(current)
    print(f"Misplaced tiles: {misplaced}")
    print(f"h(n) = {current_h}\n")

    if current_h == 0:
        print("Goal state reached!")
        return current
```

Code (continued):

```
    while True:
        step += 1
        print(f"Step {step}: Generate Neighbor States")
        neighbors = get_neighbors(current)
        evaluated = []
        for name, neighbor_state in neighbors:
            nh, n_misplaced = calculate_heuristic(neighbor_state, goal_state)
            evaluated.append((nh, name, neighbor_state, n_misplaced))
            print(f"Move {name}:")
            print_state(neighbor_state)
            print(f"Misplaced tiles: {n_misplaced}")
            print(f"h(n) = {nh}\n")

        best = None
        for item in evaluated:
            if best is None or item[0] < best[0]:
                best = item

        best_h, best_name, best_state, best_misplaced = best
        print(f"Chosen state: Move {best_name} because h(n) = {best_h}\n")

        if best_h >= current_h:
            print(f"h(best) >= h(current) ({best_h} >= {current_h})")
            print("Search stopped: Cannot find a neighbor with lower heuristic.")
            if current == goal_state:
                print("Goal state reached!")
            else:
                print("Goal state not reached (local minimum or plateau).")
            return current

        current = best_state
        current_h = best_h
        print("Current state:")
        print_state(current)
        print(f"h(n) = {current_h}\n")

        if current == goal_state:
            print("Goal state reached!")
            return current

def get_state(prompt):
    print(prompt)
    values = []
    while len(values) < 9:
        line = input().strip()
        if not line:
            continue
        for token in line.split():
            values.append(0 if token in ("_", "0") else int(token))
    return [values[0:3], values[3:6], values[6:9]]

def main():
    initial_state = get_state("Enter initial state (3 rows, space-separated, use _ or 0 for blank):")
    goal_state = get_state("Enter goal state (3 rows, space-separated, use _ or 0 for blank):")
    hill_climbing(initial_state, goal_state)

if __name__ == "__main__":
    main()
```

Output:

```
(base) sanjaywaradkar@Sanjays-MacBook-Air-2 AI_Lab % python3 exp9/exp9.py
Enter initial state (3 rows, space-separated, use _ or 0 for blank):
1 2 3
_ 4 6
7 5 8
Enter goal state (3 rows, space-separated, use _ or 0 for blank):
1 2 3
4 5 6
7 8 _
Initial State:
1 2 3
_ 4 6
7 5 8
Misplaced tiles: [4, 5, 8]
h(n) = 3

Step 1: Generate Neighbor States
Move Up:
_ 2 3
1 4 6
7 5 8
Misplaced tiles: [1, 4, 5, 8]
h(n) = 4

Move Down:
1 2 3
7 4 6
_ 5 8
Misplaced tiles: [7, 4, 5, 8]
h(n) = 4

Move Right:
1 2 3
4 _ 6
7 5 8
Misplaced tiles: [5, 8]
h(n) = 2

Chosen state: Move Right because h(n) = 2

Current state:
1 2 3
4 _ 6
7 5 8
h(n) = 2

Step 2: Generate Neighbor States
Move Up:
1 _ 3
4 2 6
7 5 8
Misplaced tiles: [2, 5, 8]
h(n) = 3

Move Down:
1 2 3
4 5 6
7 _ 8
Misplaced tiles: [8]
h(n) = 1
```

Output (continued):

```
Move Left:
1 2 3
_ 4 6
7 5 8
Misplaced tiles: [4, 5, 8]
h(n) = 3

Move Right:
1 2 3
4 6 _
7 5 8
Misplaced tiles: [6, 5, 8]
h(n) = 3

Chosen state: Move Down because h(n) = 1

Current state:
1 2 3
4 5 6
7 _ 8
h(n) = 1

Step 3: Generate Neighbor States
Move Up:
1 2 3
4 _ 6
7 5 8
Misplaced tiles: [5, 8]
h(n) = 2

Move Left:
1 2 3
4 5 6
_ 7 8
Misplaced tiles: [7, 8]
h(n) = 2

Move Right:
1 2 3
4 5 6
7 8 _
Misplaced tiles: []
h(n) = 0

Chosen state: Move Right because h(n) = 0

Current state:
1 2 3
4 5 6
7 8 _
h(n) = 0

Goal state reached!
```